

SDP



Copilot Chat Internals

Debugging AI Interactions

Transform AI debugging from guesswork into systematic investigation

"Why can't I reproduce this Copilot failure?"

When AI responses break, most engineers have nowhere to look. Copilot now ships a full debugging stack — if you know where to find it.

Platform Engineers

Debug agent pipelines and MCP configurations before users notice

Developers

Understand why context isn't loading or responses are wrong

DevOps / SRE

Trace reliability issues back to model turns, not infrastructure

45 min → 4 min

Typical config-error diagnosis time with vs. without the debug view

Black box → Observable

Agent Debug Panel exposes every model turn and tool call in real time

v1.110 — Feb 2026

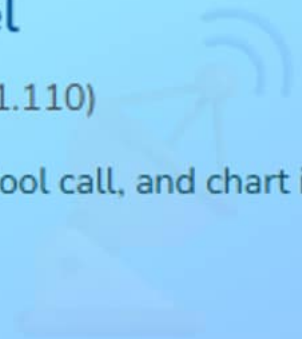
Real-time event stream and chart visualization newly available

PART 1

Agent Debug Panel

Real-Time Event Stream (v1.110)

Monitor every model turn, tool call, and chart in real time

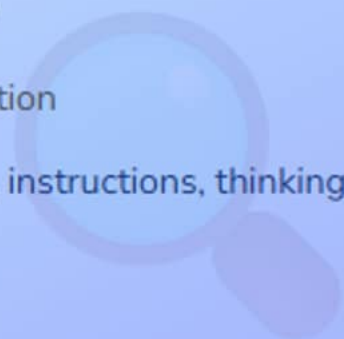


PART 2

Chat Debug View

Complete Request Inspection

See the full pipeline: files, instructions, thinking tokens

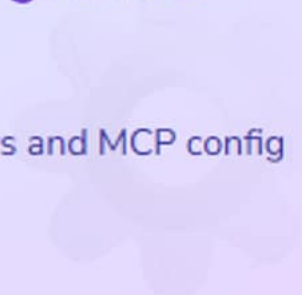


PART 3

Customization Diagnostics

Configuration Validation

Diagnose instruction errors and MCP config issues fast



PART 4

Troubleshooting Patterns

Systematic Investigation

Repeatable triage workflows to apply the same day



Click any section to jump directly there

Agent Debug Panel: Real-Time Events

Live event stream and chart visualization — Copilot goes from black box to observable system



Real-Time Events

Monitor tool calls, model turns, and errors as they happen



Chart Visualization

Timeline view of agent execution — spot bottlenecks instantly



Tool Call Inspection

Expand any event to see inputs, outputs, and durations

Opening the Agent Debug Panel in VS Code (v1.110+)
↳ Every model turn and tool call – visible in real time

Four Event Streams — All Live



Chat Events

Every message, tool call, and response in chronological order



Tool Invocations

Live timing data for each tool call — instant bottleneck detection



Customization Loading

Which instruction files, skills, and agents loaded — with status



Errors and Warnings

Immediate visibility into failures with context and error detail



Keep the panel open during development sessions — your AI interaction dashboard.

Caught: Custom Instructions Not Loading

From 35 minutes of guesswork to 90 seconds in the panel

```
$ Developer: Open Agent Debug Panel
> "Why is Copilot ignoring my custom instructions?"
📁 Loaded Customizations:
  .github/agents/backend.md - loaded
  .github/copilot-instructions.md - YAML syntax error line 14
  .vscode/settings.json - empty file, skipped
```

```
✅ Root cause found in 90 seconds
✅ Fix: corrected YAML frontmatter - instructions now apply
```

35 min → 90 sec to root cause

Two Lenses on Every Debugging Session

Chart View

Flow Diagram

Agent execution as a visual graph

Timing Bottlenecks

Spot slow tool calls at a glance

Tool Call Chains

Follow execution path through steps

Switch: list view to chart view

Loaded Customizations

Per-file status

Loaded — Warning — Failed with reason

File location

Exact path and scope shown

Application order

Which instructions apply first

First stop for missing instructions

From Reload-and-Pray to Systematic Debugging

The panel replaces guesswork with evidence

✗ Old Habit: Reload and Guess

- 1 Instructions ignored
Response does not match expectations
- 2 Reload the window
Hope the problem disappears
- 3 Try the same prompt
Same wrong result appears again
- 4 Add context manually
30+ minutes — root cause unknown

🔄 30 minutes of trial and error — no root cause

✓ New Pattern: Debug Panel First

- 1 Instructions ignored
Response does not match expectations
- 2 Open Agent Debug Panel
Developer: Open Agent Debug Panel
- 3 Check Loaded Customizations
See the failed file and error message
- 4 Fix syntax error, verify
Instructions load correctly in 2 minutes

✓ Root cause found and fixed in under 2 minutes

30 min → 2 min — panel eliminates trial-and-error debugging

Chat Debug View: Full Pipeline

Unify Chat Debug View, Thinking Tokens, and Context Management in one pipeline mental model



Request Inspector

See every file, instruction, and tool call sent to the model



Thinking Tokens

Watch model reasoning steps before the response arrives



Context Management

Control compaction and window limits for agent sessions

Opening Chat Debug View during an agent session
↳ Full request pipeline – nothing hidden from you

The 5-Stage Request Pipeline

Nothing is hidden — Chat Debug View captures every stage of every request

Prompt	Your request plus active file and any @-referenced context attached	User input
Assembly	Instructions, skills, agents, and tools merged into one payload	Context build
System prompt	Final merged payload sent as the model instruction layer	LLM input
Inference	Model generates response, calling tools as needed in real time	Tool calls
Response	Formatted output delivered — Debug View captures every stage above	Full trace

Knowing which stage failed cuts diagnosis time from hours to minutes

Context Inspection: Finding the Wrong File

Model was correct — the context was wrong

```
$ Developer: Show Chat Debug View
> "Response references the wrong version of AuthService"
🔍 Chat Debug View — Context:
  Context included:
  AuthService.v1.ts (120 tokens)
  AuthService.ts — not included
  Tool invocations:
  search_codebase returned v1 file path

✅ Context included the wrong file — not a model error
✅ Fix: attach #file:AuthService.ts explicitly
```

Debug view shifts blame from model to context — usually correct

What Thinking Tokens Reveal



Problem Decomposition

How the model breaks down complex requests into subtasks



Tool Selection Logic

Why specific tools were chosen over alternative approaches



Context Evaluation

How files and instructions shaped the model decisions



Uncertainty Signals

When the model considers multiple approaches before choosing



Unexpected response? Read the thinking — the model usually had a reason.

Context Window: Control Before It Controls You

Without Context Management

Long sessions produce vague, forgetful responses

Developers restart conversations and lose progress

No warning before overflow

Context fills silently until quality drops

Tool results excluded without notice

With `/compact` and `/fork`

`/compact`: 95% to 42% — key decisions preserved

Context window indicator

Visual percentage with token breakdown

`/fork`: branch at decision points without losing history

Proactive compaction before quality degrades

95% → 42%

context reduction after `/compact`

>80%

threshold to trigger `/compact`

0 lost

key decisions preserved



Use `/compact` proactively — do not wait for responses to degrade.

Customization Diagnostics

Diagnose misconfigured instructions — the top failure mode — plus MCP and extension log infra



Instruction Validator

Test custom instructions are loading and applying correctly



Extension Logs

Surface errors from MCP servers and language extensions



Config Health Check

Verify workspace settings before blaming the model

Running diagnostics on a broken agent configuration
↳ Misconfiguration caught in 4 min instead of 45

Four Failure Modes — One Tool to Catch Them

Open Diagnostics View — catch config errors before they waste an hour

Agent missing

YAML syntax error — Diagnostics shows the exact line number and file

Fix: syntax

Ignored rules

Instruction file in wrong location — must be in the .github/ folder

Fix: location

Skills miss

applyTo glob does not match file path — update the pattern in skill.md

Fix: glob

MCP silent

MCP: List Servers — Show Output reveals server crash or auth timeout

Fix: restart

Check Diagnostics View before spending 30 minutes blaming the model

Diagnostics View vs Extension Logs

Diagnostics View

Right-click in Chat

Instant access — no configuration needed

Load status per file

Loaded / Warning / Failed with reason

Application order

Which instructions apply first

Best for: config and customization problems

Extension Logs

Developer: Set Log Level

Set Trace for Copilot and Copilot Chat

Network and auth state

Connectivity, proxy, certificate issues

MCP: List Servers

Status, output logs, restart controls

Best for: network, auth, and infrastructure

Systematic Troubleshooting

Repeatable triage workflows and decision trees practitioners can apply the same day



Triage Workflow

Start with observability, then dig into the request layer



Decision Tree

Route symptom → root cause → fix in three steps



Evidence Checklist

Capture the right data before opening a bug report

Applying systematic debugging to a real agent failure
↳ From symptom to fix in under 5 minutes

Four Symptoms — Four Diagnostic Paths

Route the problem to the right tool without guesswork

Bad response

Open Chat Debug View — check context sent versus context expected

Debug View

Ignored rules

Open Diagnostics View — instruction file may have a syntax error

Diagnostics

Tool failure

Expand tool invocations in Debug View — check MCP server logs

MCP Logs

Session drift

Use `/compact` when context exceeds 80% — preserves decisions

`/compact`

Systematic triage: identify the layer before applying the fix

Ad-Hoc Guessing vs Systematic Investigation

Systematic debugging finds root cause every time — guessing sometimes gets lucky

✗ Ad-Hoc Debugging

- 1 Strange response
Instructions seem wrong or missing
- 2 Reload, rephrase, hope
Same problem appears again
- 3 Give up, re-add context
Context re-added manually every session
- 4 Problem recurs
No root cause identified — cycle repeats

🔄 Problem recurs — no root cause identified

✓ Systematic Investigation

- 1 Strange response
Instructions seem wrong or missing
- 2 Open Agent Debug Panel
Check Loaded Customizations in 10 seconds
- 3 Open Chat Debug View
Verify context or check Diagnostics
- 4 Fix root cause once
Syntax error, file path, or glob pattern

✓ Fixed permanently in 4 minutes

Systematic debugging: find root cause in minutes, not hours

The Debug Stack Ships With Your Editor

You had the tools already — now you know where to find them

4 min

to diagnose a config error that took 45 minutes
before

VS Code diagnostics view — real practitioner time reduction

🔍 Agent Debug Panel

Real-time event stream catches load failures instantly

🔍 Chat Debug View

Request pipeline inspection eliminates context guesswork

⚙️ Diagnostics View

Config validation before 30 minutes of troubleshooting

📄 Extension Logs

Trace mode surfaces network and auth root causes

💡 The debug stack is already in your editor — the only thing missing was knowing where to look.

From Guesswork to Systematic AI Debugging

Before

Guess at why Copilot responded incorrectly

No visibility into what context was sent to the model

Config errors take 30–45 minutes to diagnose

Reproduce AI failures by trial and error

After

Agent Debug Panel shows every event as it happens

Chat Debug View exposes all files, instructions, and tool calls

Config diagnostics catch instruction errors in minutes

Thinking tokens reveal model reasoning before the response

45 min → 4 min

Config error diagnosis time

100% visibility

Chat Debug View exposes full request pipeline

v1.110

Agent Debug Panel — Feb 2026

✓ What You Can Do Today

Today

- ❑ Open Agent Debug Panel in VS Code (v1.110+)
- ❑ Enable Chat Debug View for your current agent session
- ❑ Check one instruction file through the diagnostics panel

This Week

- ❑ Add debug view to your standard agent troubleshooting flow
- ❑ Explore thinking tokens on a failing agent task
- ❑ Validate all custom instruction files in your workspace

This Month

- ❑ Build a team triage checklist using systematic patterns
- ❑ Profile context usage to optimize agent performance
- ❑ Document common failure signatures for your agent setup

🔑 Key Takeaway

The debug stack is already in your editor — the only thing missing was knowing where to look.

 Official Documentation

[VS Code release notes: February 2026 \(v1.110\)](#)

Agent Debug Panel and real-time event stream

[Copilot Chat debug view](#)

Complete request pipeline inspection

[Copilot troubleshooting guide](#)

Systematic diagnostics and common fixes

[Context compaction documentation](#)

/compact and context window management

[Custom instructions in VS Code](#)

Write and validate customization files

 Related Content

[MCP server configuration](#)

Configure and debug external tool servers

[Copilot Memory](#)

Context persistence across agent sessions

[MCP Apps](#)

Debugging MCP server integrations



Copilot Chat Internals

Debugging AI Interactions

Agent Debug Panel

Real-time event stream and chart visualization — v1.110

45 min → 4 min

Config-error diagnosis time with the Chat Debug View

Zero hidden requests

Every file, instruction, and tool call fully visible

What's your most common Copilot debugging pain point right now?